AI-Assisted Coding
Lab Assignment - 13.4

Name: Zobiya Fatima
Roll No: 2303A51879
Batch No: 14

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions
Task 1: Refactoring Data Transformation Logic
Scenario
You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.
Legacy Code

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Expected Output
[4, 8, 12, 16, 20]

Code Before Refactoring:

```
Modify selected code
Add Context...
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
doubled = [v * 2 for v in values]
print(doubled)
```

Code After Refactoring:

```
values = [2, 4, 6, 8, 10]
doubled = [v * 2 for v in values]
print(doubled)
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-13/task-1.py"
[4, 8, 12, 16, 20]
PS C:\AI Assisted Coding\LAB>
```

Explanation:

- The list comprehension [v * 2 for v in values] iterates through each element in values.
- Each value is multiplied by 2.
- The results are directly stored in the new list doubled.
- This preserves the same output as the original loop but in a more Pythonic way.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word by word.

Legacy Code

words = ["Refactoring", "with", "AI", "improves",
"quality"]
message = ""
for w in words:
message += w + " "
print(message.strip())

Expected Output

Refactoring with AI improves quality

Code Before Refactoring:

```
Modify selected code
Add Context...
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
message = " ".join(words)
print(message)
```

Code After Refactoring:

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = " ".join(words)
print(message)
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-13/task-2.py"
Refactoring with AI improves quality
PS C:\AI Assisted Coding\LAB>
```

Explanation

- " ".join(words) combines all the words in the list into a single string.
- The " " (space) is placed between each word automatically.
- This method avoids repeated string concatenation and makes the code cleaner and faster while producing the same output.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
print(config["timeout"])
else:
print("Default timeout used")
```

Expected Output

Default timeout used

Code Before Refactoring:

```
Modify selected code
Add Context...
4  config = {"host": "localhost", "port": 8080}
   if "timeout" in config:
       print(config["timeout"])
   else:
       print("Default timeout used")
5  print(config.get("timeout", "Default timeout used"))
6
```

Code After Refactoring:

```
4  config = {"host": "localhost", "port": 8080}
5  print(config.get("timeout", "Default timeout used"))
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-13/task-3.py"
Default timeout used
PS C:\AI Assisted Coding\LAB>
```

Explanation

- config.get("timeout", "Default timeout used") tries to retrieve the value for "timeout".
- If the key **exists**, it returns its value.
- If the key **does not exist**, it returns the **default message** "Default timeout used".
- This approach removes the need for an if-else check and makes the code **shorter, safer, and more readable**.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Legacy Code

action = "divide"
x, y = 10, 2
if action == "add":
result = x + y
elif action == "subtract":
result = x - y
elif action == "multiply":
result = x * y
elif action == "divide":
result = x / y
else:
result = None
print(result)

Expected Output

5.0

Code Before Refactoring:

```python
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None

operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b,
}

result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

Code After Refactoring:

```python
action = "divide"
x, y = 10, 2

operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b,
}

result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-13/task-4.py"
5.0
PS C:\AI Assisted Coding\LAB>
```

Explanation

- A **dictionary (operations)** maps operation names to functions.
- operations.get(action, lambda a, b: None) retrieves the corresponding function.
- If the action is not found, it returns a default function that returns None.
- The selected function is then executed with (x, y).
- This approach is **cleaner, easier to extend, and more scalable** than multiple if–elif statements.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Legacy Code

inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
if item == "eraser":
found = True
break
print("Item Available" if found else "Item Not Available")

Expected Output

Item Available

Code Before Refactoring:

```
Modify selected code
Add Context...
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
    break
print("Item Available" if found else "Item Not Available")



# Refactored version using 'in' operator
found = "eraser" in inventory
print("Item Available" if found else "Item Not Available")
```

Code After Refactoring:

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
    break
print("Item Available" if found else "Item Not Available")

# Refactored version using 'in' operator
found = "eraser" in inventory
print("Item Available" if found else "Item Not Available")
```

Output:

```
Item Available
PS C:\AI Assisted Coding\LAB>
```

Explanation

- The **in operator** directly checks whether "eraser" exists in the inventory list.
- It replaces the manual loop and flag variable (found).
- This makes the code **shorter, clearer, and easier to maintain** while keeping the same output.